TECHNICAL ARCHITECTURE SPECIFICATION

Backend Architecture — Anonymous Men's Support Platform

Backend Design Document · Privacy-First Architecture

1. Guiding Constraints

This isn't a generic booking-app backend. Two things shape every decision below:

- **Anonymity is a hard requirement, not a feature.** Real identity (name, email, phone, payment method) must never be reachable from the same data path as counseling/session data. If those two ever join in one query, the anonymity guarantee is broken.
- **The data is sensitive-category data** (mental health, personal struggles). Treat it like health data even if not strictly regulated in your jurisdiction — encryption at rest/in transit, strict access control, audit trails, and data minimization throughout.

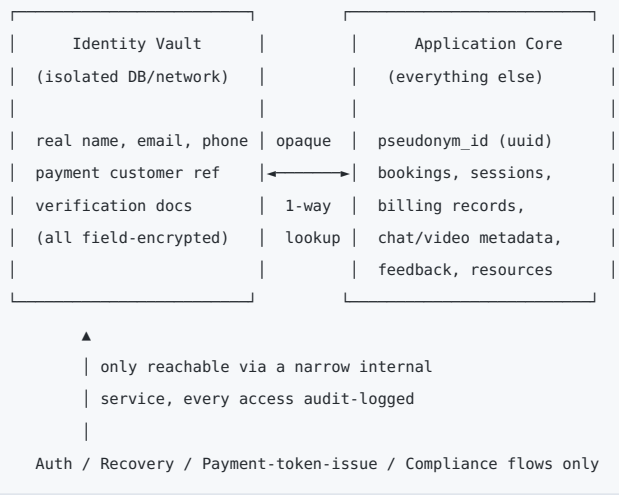
Everything else (booking, billing, video) is comparatively standard and shouldn't be over-engineered.

2. Recommended Stack

Concern	Choice	Why
Language	TypeScript	Type safety matters when sensitive fields flow through many layers
Framework	NestJS	Modular by design, DI, guards/interceptors map well to auth & encryption boundaries
Primary DB	PostgreSQL	ACID for bookings/billing, row-level security for tenant isolation
Cache/queue	Redis + BullMQ	Sessions, rate limiting, presence, background jobs
Object storage	S3-compatible, server-side encrypted	Resource library assets, never session content by default
ORM	Prisma	Clear schema, good migration story
Real-time	Socket.IO (chat/presence) + managed WebRTC (Daily.co / Twilio Video / Agora)	Building your own SFU (mediasoup/Janus) is a valid path later, but not for MVP
Payments	M-Pesa (Safaricom Daraja API) now, Pesapal later — behind a common gateway abstraction	M-Pesa covers the primary market; Pesapal adds cards/other rails later without touching billing logic
Secrets/keys	AWS KMS or HashiCorp Vault	Envelope encryption for PII fields
Infra	Docker → ECS/Fargate or Kubernetes later	Start simple
IaC/CI	Terraform + GitHub Actions	
Observability	Pino (with redaction) + Prometheus/Grafana + Sentry (PII-scrubbed)	

Start as a **modular monolith** (NestJS modules with strict boundaries), not microservices. The one exception is the **Identity Vault** (below) — that should be network-isolated from day one even if it's just a separate schema/DB instance, because retrofitting that isolation later is painful.

3. The Core Idea: Identity Vault Separation



- Every user (client, provider, admin) is addressed everywhere in the application core by a **pseudonym_id (UUID)**. No module outside the vault ever sees real name/email/phone.
- The vault is the only place PII lives, and only a narrow internal service (`IdentityVaultService`) can query it — every read is audit-logged with actor + reason.
- Providers are also pseudonymous to clients. Real credential verification (license checks, background checks) happens once during onboarding via the vault + a compliance/admin flow, not exposed elsewhere.
- Payments: phone numbers used for M-Pesa (and any card/identity details Pesapal collects later) live in the vault; billing records in the app core reference only the pseudonym + an opaque transaction reference (M-Pesa `CheckoutRequestID`, Pesapal `OrderTrackingId`). Safaricom/Pesapal will see a real phone number/name at the payment-rail level (unavoidable — mobile money is inherently tied to a SIM-registered identity) — that's an accepted boundary, isolated from counseling data the same way Stripe would have been.
- Recovery (password/account reset) is the one flow allowed to bridge vault ↔ pseudonym, and it should require step-up verification.

4. Module Breakdown (NestJS modules)

- **auth** — pseudonymous signup/login (username + password, or OTP via vault-held contact info), JWT access + refresh tokens, MFA optional
- **identity-vault** — isolated module/service as described above; separate DB connection, separate deploy target if possible
- **users** — pseudonymous profile records (role, status, preferences)
- **providers** — provider profiles, specialties, availability, verification status (verification *result* only — not the documents, which stay in the vault)
- **matching** — matches clients to providers/support groups by need, specialty, availability
- **booking** — scheduling, session lifecycle (requested → confirmed → completed/cancelled)
- **sessions** — orchestrates chat/video: creates ephemeral rooms with the video provider, issues short-lived join tokens, tracks *metadata only* (start/end time, duration, channel type) — never message/video content
- **billing** — payment gateway abstraction (M-Pesa now, Pesapal later): minimum 30-min billing, hourly overage after, subscription plans, provider payouts (see §11)
- **notifications** — email/SMS/push; content must stay generic ("You have a new session reminder" — never "your therapy session"), since notification delivery itself can leak sensitive context to email/SMS providers or a shared inbox/lock screen
- **support-groups** — anonymous group sessions, membership by pseudonym
- **resources** — content library (articles/videos), public or gated
- **feedback** — post-session ratings tied to pseudonym + session, not identity
- **admin** — moderation, provider verification approval, break-glass access to vault (requires justification, logged, ideally dual-control)
- **audit** — centralized audit log for all sensitive actions, especially vault access

Shared cross-cutting concerns (`common/`): encryption utils (AES-256-GCM field encryption), RBAC/ABAC guards, request-context middleware for audit logging, global exception filter that scrubs PII from error output.

5. Data Model (key entities, simplified)

```
User (app core)            IdentityRecord (vault)
 ├ id (uuid, pseudonym)    ├ pseudonym_id (fk, uuid)
 ├ role                    ├ encrypted_name
 ├ status                  ├ encrypted_email
 └ createdAt               ├ encrypted_phone
                           ├ mpesa_customer_ref (encrypted)
                           └ verification_status

ProviderProfile            ClientProfile
 ├ userId (pseudonym)      ├ userId (pseudonym)
 ├ displayName (pseudonym) ├ preferences
 ├ specialties[]           └ intake_notes (encrypted, optional)
 └ availability

Booking                    Session
 ├ clientId, providerId    ├ bookingId
 ├ scheduledStart          ├ channelType (chat/video)
 ├ durationMin             ├ startedAt / endedAt
 ├ billingType             └ (NO content stored)
 └ status

Payment                    Subscription
 ├ userId (pseudonym)      ├ userId (pseudonym)
 ├ provider (enum)         ├ plan
 ├ externalRef             ├ sessionsIncluded / renewalDate
 │  (CheckoutRequestID /   └ status (active/lapsed —
 │   OrderTrackingId)           see §11 on recurring caveats)
 ├ amount, status
 └ direction (charge/payout)

Feedback                   AuditLog
 ├ sessionId               ├ actorPseudonym
 ├ rating, comment         ├ action, target
 └ (no identity link)      └ timestamp
```

6. Security Baseline

- TLS everywhere, HSTS, strict CSP on any web client
- Field-level encryption (AES-256-GCM, envelope-encrypted via KMS) for every PII field in the vault
- Session content: video encrypted in transit via the provider's SRTP/DTLS; chat should be end-to-end encrypted (client-held keys) if you store any message history at all — simplest safe default is **don't persist chat content server-side**, only metadata
- RBAC + ABAC guards: providers can never query client identity; admins get scoped, time-limited, audited access to the vault only
- Rate limiting + brute-force protection on auth (Redis-backed)
- Data retention policy + right-to-delete flow (delete vault record + cascade-anonymize app-core records)
- No production PII in logs — redact at the logger level (Pino redaction paths), scrub Sentry payloads

7. Folder Structure

```
apps/
  api/                    # main NestJS app
    src/
      modules/
        auth/
        identity-vault/   # isolated module, own DB connection
        users/
        providers/
        matching/
        booking/
        sessions/
        billing/
        notifications/
        support-groups/
        resources/
        feedback/
        admin/
        audit/
      common/
        guards/
        interceptors/
        encryption/
        filters/
      config/
  worker/                 # BullMQ job processor (notifications, billing reconciliation, matching)
libs/
  shared/                 # shared DTOs/types between api & worker
infra/
  docker/
  terraform/
  k8s/                    # if/when you outgrow ECS/Fargate
```

8. Phased Roadmap

- **Phase 1 (MVP):** pseudonymous auth, identity vault, client/provider profiles, booking, one video provider integration, M-Pesa billing (STK Push, minimum + hourly), minimal notifications
- **Phase 2:** support groups, resource library, feedback/ratings, matching algorithm
- **Phase 3:** admin/moderation tooling, provider verification workflow, privacy-preserving analytics, mobile push

9. Admin Panel

The admin panel is a separate frontend against the same `admin` module's API surface, gated by role + (ideally) a stricter auth path than the regular app (MFA required, IP allowlist optional). It should NOT be a superset of the client/provider dashboards bolted together — it needs its own RBAC scopes so a support agent can't do what a compliance officer can.

Roles (minimum viable split):

- **Support agent** — booking/session troubleshooting, no vault access
- **Moderator** — content/abuse handling, feedback moderation, no vault access
- **Compliance officer** — provider verification, break-glass vault access (logged, dual-control)
- **Super admin** — role management, system config

Screens/capabilities:

Area	What it does
Provider verification queue	Review submitted credentials/licenses, approve/reject/request more info (detail in §10)
User & provider management	View pseudonymous accounts, suspend/ban, reinstate, view status history
Booking & session oversight	Monitor session health, no-shows, cancellations, disputes; force-cancel/refund a session
Billing & subscriptions	Refunds, plan overrides, dispute handling, failed-STK-push retries — operates on pseudonym + transaction ref, not real identity
Content moderation	Feedback comments, support-group reports, abuse flags, resource library publishing
Support group management	Create/schedule groups, assign/rotate moderators, cap group size
Resource library CMS	Author/edit/publish articles & videos
Analytics dashboard	Aggregate-only metrics (session volume, ratings, retention, provider load) — no individual drill-down by default
Audit log viewer	Every vault access and admin action: actor, target, reason, timestamp
Break-glass vault access	Time-boxed, justification-required, dual-control-if-possible access to real identity (legal request, safety escalation, payment dispute)
Broadcast/system notices	Platform-wide announcements, incident notices

Two things worth being strict about from day one: (1) every admin action that touches a user record — suspend, refund, vault access — writes to `audit`, no exceptions; (2) analytics should be built off aggregated/anonymized read models, not ad-hoc queries against live pseudonym-linked tables, so "just checking the numbers" can never accidentally become "just looking up a specific guy."

10. Provider (Therapist/Coach) Registration & Request Handling

Important nuance: **anonymity in this system is client-facing, not platform-facing.** Providers are licensed professionals — the platform has a real accountability and liability need to know exactly who they are, verify credentials, and be able to act if something goes wrong. What providers get is anonymity *from clients*, not from the platform. Practically: real identity + credentials go in the Identity Vault like everyone else; what's exposed publicly is a pseudonymous provider profile.

10a. Registration / onboarding flow

```
1. Sign up           → pseudonym_id created, contact info → vault (pending status)
2. Credential intake → license #, certifications, ID, malpractice insurance (if applicable)
                       uploaded to encrypted intake storage, linked to vault record
3. Review queue      → compliance officer verifies license against issuing body/registry,
                       reviews documents, approves/rejects/requests more info
4. Approved          → provider creates public pseudonymous profile (display name,
                       specialties, bio, rate card) — this is the only thing clients see
5. Active            → provider sets availability, starts receiving bookings
```

A `ProviderVerification` entity sits adjacent to the vault (license number, document refs, verifying body, expiry date, reviewer, decision timestamp) — encrypted, admin/compliance-only access, audit-logged same as vault reads. License expiry should drive a renewal reminder workflow, not silently lapse.

10b. How requests reach providers — recommend a hybrid model

Two patterns solve different needs; use both rather than picking one:

- **Direct booking (default for 1:1 coaching/counseling):** client browses pseudonymous provider profiles (specialty, bio, rating, published availability) and books an open slot directly, like Calendly. Slot reserves instantly on booking — no accept/decline round trip needed since the provider already published that slot as open. Best UX, and lets clients choose a "fit" without needing real identity.

- **Auto-match / request queue (for support groups, crisis-adjacent, or "just get me someone now"):** client submits a request (need/topic, time window) instead of picking a person; the matching engine assigns it to the best-available provider by specialty + load + availability. That provider gets a request they can accept/decline within a short window (e.g., 15 min) before it reassigns. Better for speed and for balancing load across providers than making an anxious user pick a stranger's profile.

10c. Provider-side experience

Providers get their own dashboard (role-gated view in the same app, not a separate product):

- Incoming requests (queue model) / upcoming confirmed sessions (direct-book model)
- Availability/calendar management
- Session history, client feedback (providers see client pseudonym only — same anonymity boundary in reverse)
- Earnings & payout status

Delivery of "you have a new request" should go over a private WebSocket channel per provider for live dashboard updates, backed by a push/email fallback with deliberately generic copy ("New session request — open the app") since email/push payloads sit outside your encryption boundary (phone lock screens, third-party mail servers).

11. Payments: M-Pesa Now, Pesapal Later

M-Pesa (via Safaricom's Daraja API) behaves differently from a card processor like Stripe in ways that actually shape the billing module: it's **push-based and asynchronous** (you initiate a prompt, the customer approves on their phone, the result arrives later via callback), and it has **no native recurring-billing primitive**. Design for that from the start so adding Pesapal later is a config change, not a rewrite.

11a. Gateway abstraction

Define a `PaymentGateway` interface in the `billing` module (`initiateCharge`, `handleCallback`, `queryStatus`, `payout`) with an `MpesaGateway` implementation now and a `PesapalGateway` implementation later. Nothing else in the codebase — booking, subscriptions, refund logic — should know which rail is behind it. The `Payment.provider` field (from §5's data model) is the only thing that varies.

11b. M-Pesa (Daraja) integration specifics

- **STK Push (Lipa na M-Pesa Online)** for client charges: your server requests Safaricom to prompt the customer's phone; customer enters PIN; result comes back **asynchronously** to a callback URL you register — not in the initiating response. This means the booking/payment record must start in a `pending` state and transition on callback, not on the initial API response.
- **OAuth token caching:** Daraja access tokens expire hourly — cache in Redis, refresh proactively, don't fetch per-request.
- **Idempotency:** every STK push returns a `CheckoutRequestID` / `MerchantRequestID` — use it as the idempotency key, since callbacks can arrive more than once or be delayed.
- **Reconciliation job (important):** callbacks occasionally never arrive (network drop, user closes the prompt). Run a BullMQ job that polls the **STK Push Query API** for any payment still `pending` after N minutes, so a real transaction doesn't get silently stuck. Don't rely on the callback alone.
- **Provider payouts (B2C):** paying providers out is a separate Daraja product (Business-to-Customer) with its own credentialing (security credential encrypted against Safaricom's public cert) and its own approval process with Safaricom — budget onboarding lead time for this, it's slower than getting C2B/STK push approved.
- **Infra requirement:** the callback URL must be a publicly reachable HTTPS endpoint, including during development/certification with Safaricom — plan for a stable tunnel (ngrok or similar) or a real staging domain early, it'll block your Daraja sandbox-to-production certification otherwise.

11c. Subscriptions without native recurring billing

Stripe-style "charge automatically every month" doesn't exist on M-Pesa. Two workable patterns:

- **Re-prompt monthly:** a scheduled job sends a fresh STK push a few days before renewal; subscription lapses to a grace period if the customer doesn't approve it in time. Simple, but it's an active action from the user every cycle, not silent.
- **Defer true auto-renewal to Pesapal/cards:** Pesapal (and card rails generally) support tokenized recurring charges properly. If subscriptions matter a lot for retention, that's a reason to pull Pesapal forward instead of treating it as strictly "later."

Given the concept note's subscription model, I'd ship Phase 1 with pay-per-session M-Pesa billing (matches the minimum-billing/hourly model directly) and treat monthly subscriptions as a Phase 2 item gated on whichever of the two patterns above you pick.

11d. Pesapal (Phase 2)

Pesapal is an aggregator (M-Pesa + cards + other mobile money) with a redirect/iframe checkout and its own callback (IPN) model — conceptually similar async shape to M-Pesa, so the `PaymentGateway` abstraction holds up. Main reason to add it later: cards for clients outside Kenya, and (per §11c) real recurring billing if you want that before building the manual re-prompt flow.

12. Open Decisions Worth Revisiting

- Video provider: managed (Daily.co/Twilio/Agora — faster, less ops) vs. self-hosted SFU (mediasoup — full data control, more engineering)
- Whether any session content is ever persisted (even encrypted) for QA/compliance, or strictly metadata-only
- Jurisdiction-specific compliance target (GDPR-style vs. HIPAA-adjacent vs. none formally, but "act like it" regardless)
- Direct-booking vs. auto-match as the *default* for standard 1:1 sessions (recommended above, but worth confirming against your matching-quality expectations)
- How rigorous provider credential verification needs to be at launch (self-attested vs. licensing-body verification vs. third-party background-check service) — affects both trust and time-to-launch